Übungsblatt 5: Inter-Task-Kommunikation und Interrupts

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Ziele und Ablauf

Sie wählen einen Kommunikationsvertrag, verfolgen Queue-Belegung, prüfen Pufferbesitz und rechnen eine Interruptübergabe nach. Grundlage sind Vorlesungen 1–5. FIFO bedeutet First In, First Out: Entnahme in Einfügereihenfolge.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	30 Orientierungspunkte
Transfer und Besprechung	20	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **85 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten. Planen Sie im Papierkern 20 Minuten für Aufgabe 1, 20 Minuten für Aufgaben 2–3 und 10 Minuten für Aufgabe 4.

Aufgabe 1: Ring und Wartezustände · 8 Punkte

Lehrkonvention: FIFO, $N = 3$, Head ist der nächste Entnahmeplatz, Tail der nächste Einfügeplatz, Count die Belegung. Alle drei Plätze sind nutzbar. Start: Head = Tail = Count = 0. Ein volles Try-Put verwirft den neuen Wert und ändert keine Queue-Daten. Ein leeres Try-Get liefert keine neue Nachricht. Try bedeutet Timeout null, ohne Warten.

1. Verfolgen Sie Put 10, Put 20, Get, Put 30 und Put 40. Geben Sie nach jedem Schritt logische Folge, Head, Tail und Count an. Geben Sie danach die physische Slotbelegung an. (3 P)
2. Setzen Sie fort: Put 50, Get, Put 50, Get, Get, Get, Get. Geben Sie je Operation Erfolg oder Fehler, einen gegebenenfalls gelieferten Wert und den neuen Zustand an. (3 P)
3. Welche zwei Belegungen können bei Head = Tail auftreten? Erklären Sie außerdem den Unterschied zwischen „Get wartet auf Daten“, „Put macht den Consumer Ready“ und „Consumer verarbeitet“. (2 P)

Aufgabe 2: Überlauf und Kapazität · 8 Punkte

Lehrmodell, keine Messung: Queue anfangs leer, $N = 10$. Ein Put bei jeder ganzen Millisekunde von 1 bis 20 versucht den Wert seines Zeitpunkts einzureihen. Get bei 5, 10, 15 und 20 ms entnimmt je ein Element. Zeitgleich gilt **Put vor Get**. API-Zeit ist vernachlässigt. Volle Queue verwirft den neuen Datensatz.

1. Bestimmen Sie Belegung nach 5 und 10 ms und den ersten Fehl-Put. (2 P)
2. Bestimmen Sie verworfene Werte, entnommene Werte, Zahl erfolgreicher Puts und Restbelegung bei 20 ms. Prüfen Sie beide Nachrichtenbilanzen. (3 P)
3. Wie groß müsste die Queue sein, um nur dieses begrenzte Intervall verlustfrei zu speichern? Begründen Sie die höchste Zwischenbelegung. Kann eine endliche Vergrößerung bei dauerhaft denselben Raten jeden Verlust verhindern? (2 P)
4. Wählen Sie eine begründete Überlaufregel für einen Softwareauftrag, dessen Erzeugung warten darf, und für ein Display, das nur den neuesten Zustand braucht. (1 P)

Aufgabe 3: Kopie, Besitz und Pflichtempfänger · 6 Punkte

1. Eine Wertqueue kopiert einen Datensatz mit `sample = 10`. Der Sender ändert anschließend seine lokale Kopie auf 99. Was erhält der spätere Consumer? Warum gilt dieselbe Begründung nicht für den Inhalt hinter einem kopierten Pointer? (2 P)
2. Eine Queue bietet zehn reine Pointerplätze. Es existieren zwei Bildpuffer A und B mit je 75 KiB. A wird vom Consumer gelesen, B ist erfolgreich eingereiht. Wie viele neue unabhängige Bilder kann der Producer jetzt befüllen? Berechnen Sie Pointer-Slot-Nutzspeicher bei angenommenen 4-Byte-Pointern und den tatsächlich vorhandenen Bildspeicher. (2 P)
3. Ein Pointer-Put scheitert. Wer bleibt für den Puffer verantwortlich? Reicht ein `static`-Array als Schutz vor zu frühem Wiederbefüllen? Erhalten zwei Get-Consumer derselben Queue beide jede Nachricht? (2 P)

Aufgabe 4: Interruptübergabe und Zeitbudget · 8 Punkte

1. Lehrannahmen: vier implementierte NVIC-Prioritätsbits, passende Präemptionsgruppierung, logische Kernelaufrufgrenze fünf. Darf IRQ-Priorität drei `osMessageQueuePut(q, &m, 0, 0)` aufrufen? Wie steht es um Priorität sieben? Darf `osMutexAcquire(mutex, 0)`

bei Priorität sieben aufgerufen werden? Berechnen Sie den geschobenen Registerwert der Grenze fünf und nennen Sie den Wert für den HAL-Prioritätssetter. (3 P)
2. Synthetische Spur in µs: Ereignis 0, ISR-Eintritt 3, Worker Ready 8, ISR-Ende 12, Workerstart 16, Abschluss 24. Bestimmen Sie Eintrittslatenz, ISR-Dauer, Ready-bis-Running, Startlatenz und Antwortzeit bis Abschluss. Prüfen Sie eine relative Deadline von 25 µs. Warum darf Ready-bis-Running nicht zusätzlich auf alle anderen Intervalle addiert werden? (3 P)
3. Ein anfangs leerer binärer Semaphor mit Maximum eins erhält drei Releases vor jedem Acquire. Wie viele Releases gelingen? Unterscheiden Sie diese Zusammenfassung von Tasterprellen und einem Queue-Drop. (2 P)

Transfer: Zwei Nachrichten, ein Speicher · 20 Minuten

Bearbeiten Sie den Fall zehn Minuten und besprechen Sie die Besitzregeln zehn Minuten. Keine zusätzlichen Punkte. Das folgende **abstrakte Ablaufmodell** hat keine gleichzeitigen Zugriffe: Der Consumer beginnt erst nach allen Producer-Schritten. Die Queue besitzt zwei Plätze, alle Puts gelingen.

Schritt	Producer / Queue
1	statisches Array `buf` enthält `[10, 20]`
2	Pointer auf `buf` wird eingereiht
3	Producer ändert dasselbe Array auf `[30, 40]`
4	derselbe Pointer wird nochmals eingereiht
5	Consumer entnimmt beide Pointer und liest die jeweiligen Arrays

1. Welche beiden Inhalte liest der Consumer? Warum sind zwei Queueplätze keine zwei unabhängigen Bildpuffer?
2. Vergleichen Sie mit einer Queue, die bei jedem Put die beiden Zahlen **als Werte** kopiert. Welche Inhalte werden dann entnommen?
3. Entwerfen Sie für die Pointervariante einen Besitzablauf mit zwei verschiedenen Puffern und Free-Pool. Was geschieht nach fehlgeschlagenem Put? Reicht `static` zur exklusiven Nutzung?

Praxisfall · 15 Minuten

Praxisfall 5: Einen abgeschlossenen Queue-Lauf prüfen

Papieralternative, synthetischer Diagnosefall: Eine FIFO-Wertqueue hat einen Producer, einen Consumer und keine anderen Entnahmen. Angeboten werden Sequenzen 1–20 mit Nutzwert $10 \cdot \text{seq}$. Angenommen wurden 1, 2, 3, 4, 5, 9, 13 und 17; alle anderen Puts wurden ausschließlich wegen Vollbelegung verworfen. Eine separate STOP-Nachricht folgt hinter den angenommenen Daten. Beide Done-Kennzeichen sind gesetzt, alle Daten verarbeitet und die Queue leer. Das Consumerlog enthält dieselbe Folge mit den passenden Nutzwerten.

1. Bestimmen Sie `accepted`, `dropped` und `received` und prüfen Sie beide Bilanzen. Wird STOP als Datenversuch mitgezählt?
2. Welche Endsequenzen fehlen? Kann das Consumerlog allein diese Verluste erkennen, wenn es die Gesamtzahl der Angebote nicht kennt?
3. Ein anderer angeblich gültiger Endlog nennt 1, 2, 4, 3, 5, 9, 13, 17. Ist das mit diesem Kanalvertrag vereinbar? Begründen Sie, bevor Sie die Ursache einem Scheduler zuschreiben.

Bei vorbereitetem Board: Die Lehrperson zeigt einen bereits abgeschlossenen Wertqueue-Lauf und dessen Bilanzen. Ein eigener Button-/EXTI-Versuch ist Vertiefung; er wird für die Erklärung des Queue-Vertrags nicht benötigt.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (30 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 5; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*; die in der Vorlesung und der freiwilligen Integrationsanleitung genannten Herstellerdokumente.